TP5 : Réseau et forges

Introduction

Pour ce dernier TP sur le réseau et les forges sociales, je vous propose de refaire le monde en vous livrant à un petit exercice d'édition collaborative autour de la Déclaration des Droits de l'Homme et du Citoyen de 1789.

Pourquoi ce texte, me direz-vous ? Parce que c'est un texte court, très connu, dont le copyright a expiré depuis longtemps, et dont la nature politique se prête naturellement à des polémiques bon enfant. En un mot, un candidat idéal pour explorer les subtilités de la gestion de version collaborative.

Préparations au travail en réseau

Configuration du GitLab IN2P3

Nous allons utiliser pour cette partie de la formation la forge GitLab administrée par le centre de calcul de l'IN2P3, https://gitlab.in2p3.fr .

Malgré son affiliation institutionnelle assez spécifique, cette forge est utilisable par une grande partie des personnels de l'enseignement supérieur et de la recherche, français et étrangers, grâce à son adhésion à la fédération d'identité EduGAIN dont la couverture internationale se rapproche de celle du service wifi eduroam.

Si vous avez bien suivi les mails reçus avant la formation, vous devriez avoir d'ors et déjà configuré votre compte et savoir comment vous y connecter. Si toutefois ce n'était pas le cas, voici quelques rappels utiles.

Pour vous connecter à la forge avec vos identifiants professionnels :

1. Rendez-vous sur https://gitlab.in2p3.fr/users/sign_in .
2. Cliquez sur le bouton « EduGAIN [Shibboleth] ».
3. Choisissez un organisme académique auprès duquel vous possédez un compte informatique (ex : CNRS, Université Paris-Saclay. . .).
4. Identifiez-vous de la façon habituelle.

Si c'est la première fois que vous vous connectez, un compte vous sera créé automatiquement sur la plate-forme, et vous arriverez sur la page d'accueil de GitLab. La procédure sera la même pour vous reconnecter par la suite.

Toutefois, il ne vous suffit pas de pouvoir vous connecter à l'interface web. Pour ce TP, vous aurez besoin d'échanger des *commits* avec la forge, ce qui implique que votre client git local puisse s'authentifier en votre nom auprès du serveur. Il existe plusieurs moyens de faire cela, mais malheureusement. . .

- La configuration réseau du campus de Gif-sur-Yvette ne nous permet pas d'utiliser le protocole SSH, nous sommes donc forcés d'utiliser le HTTPS.
- Comme le client git en ligne de commande n'est pas compatible avec le mode d'authentification EduGAIN/Shibboleth, vous allez devoir créer un

mot de passe spécial juste pour les besoins de l'échange réseau entre votre ordinateur et GitLab via HTTPS.

Pour configurer un tel mot de passe...

- Cliquez dans le coin supérieur droit de l'interface GitLab
- Choisissez l'option « Settings »
- Choisissez la section « Password » dans le menu à gauche
- Saisissez un mot de passe et confirmez-le, puis cliquez sur « Save password »

Cette action aura pour conséquence de vous déconnecter de GitLab. Vous pourrez vous y reconnecter de la façon décrite précédemment. Votre mot de passe HTTPS est désormais prêt à l'emploi.

Fork et clonage

Une copie de la Déclaration des Droits de l'Homme et du Citoyen est disponible sur le dépôt git https://gitlab.in2p3.fr/grasland/ddhc/ .

Pour commencez, créez-en un *fork* sous votre contrôle, en utilisant le bouton « Fork » situé en haut à droite de l'interface de GitLab, à droite du nom « ddhc ». Choisissez de créer le *fork* au sein de votre compte personnel dans l'invite qui s'affiche alors.

L'opération va prendre un petit moment (typiquement quelques secondes). Lorsqu'elle est terminée, vous devriez être redirigé automatiquement sur votre *fork* nouvellement créé.

Pour travailler dessus localement, vous allez ensuite devoir copier son contenu sur votre machine. Pour cela, vous avez besoin de son URL HTTPS, que vous pouvez obtenir de l'interface GitLab de la façon suivante :

- Cliquez sur le bouton bleu « Clone ».
- Copiez l'URL située dans la section « Clone with HTTPS ».

Une fois l'URL obtenue, créez un clone local du *fork* de la façon suivante :

```
$ cd ~/Bureau/
$ git clone <url de votre fork>
```

Connexion au dépôt officiel

Comme notre clone local dans le TP précédent, le clone du *fork* possède une *remote* pointant vers le dépôt distant qui permet d'échanger facilement des commits avec ce dernier :

```
$ cd ddhc/
$ git remote -v
```

Mais pour appliquer la méthode de travail basée sur les *forks* et les *merge requests* (appelée *GitHub flow*), il faut aussi être capable d'échanger des commits avec

le dépôt officiel, celui à partir duquel vous avez créé votre *fork*. En effet, des nouveautés vont arriver sur ce dernier au fur et à mesure du TP, et vous devrez vous synchroniser avec ces dernières.

Par convention, la *remote* associée au dépôt officiel est souvent appelée *upstream* :

```
$ git remote add upstream https://gitlab.in2p3.fr/grasland/ddhc.git
$ git fetch upstream
```

Si vous ouvrez maintenant `gitg`, vous pourrez constater que vous pouvez maintenant visualiser vos commits et branches locales, celles de votre *fork*, et celles du dépôt officiel. Tout est prêt !

Travail distribué avec le GitHub flow

Introduction

Le *GitHub flow* est une méthode de travail qui, malgré son nom, n'a rien de spécifique à la forge GitHub. Toutes les forges sociales modernes peuvent être utilisées de cette manière. Dans cette méthode, le travail s'organise autour de quatre grandes étapes :

1. Rester à jour par rapport au dépôt officiel
2. Préparer des changements dans une branche dédiée de son dépôt local
3. Publier ses changements sur son *fork* lorsqu'ils sont prêts
4. Les soumettre au dépôt officiel pour qu'ils y soient intégrés

Examinons ces quatre étapes en détail

Se synchroniser avec le dépôt officiel

Comme nous l'avons vu dans le TP précédent, il est possible de récupérer les changements du dépôt officiel avec `git fetch` :

```
$ git fetch upstream
```

Cependant, dans le cadre de cette méthode de travail, nous n'avons généralement pas besoin de nous intéresser à l'ensemble des branches de ce dépôt. Il suffit de tenir notre branche `master` à jour, ce que l'on peut faire avec `git pull` après avoir configuré `upstream/master` comme branche amont :

```
$ git checkout master
$ git branch --set-upstream-to=upstream/master
```

Par la suite, vous pourrez mettre à jour votre master via :

```
$ git checkout master
$ git pull --ff-only
```

Je conseille ici l'utilisation de `--ff-only` pour vous aider à repérer une erreur classique. Si l'on applique correctement le *GitHub flow*, la branche `master` du dépôt officiel devrait avancer de façon monotone, et votre branche `master` locale

devrait suivre son avancée. Par conséquent, si vous ne pouvez pas mettre en avance rapide, cela relève d'un comportement pathologique :

- Soit les mainteneurs du dépôt officiel ont fait diverger `master`. C'est parfois nécessaire, mais très rare, et un mainteneur responsable vous informera lorsqu'il doit en venir à de tels extrêmes.
- Soit, cas de loin le plus courant, vous avez vous-même créé une divergence, typiquement en ajoutant des commits sur votre branche `master` locale.

Dans ce dernier cas, pour vous tirer d'affaire, vous pouvez...

- Si vous le souhaitez, sauvegarder vos changements dans une nouvelle branche avec `git branch <nom>`
- Puis forcer votre branche `master` à revenir au même point que le `master` officiel avec la commande `git reset --hard upstream/master`

Construire des changements dans une branche dédiée

Une fois que l'on a un `master` à jour, il est possible de construire des changements par-dessus ce dernier. Il suffit pour cela de créer une branche qui part de master et de commencer à créer des *commits* dessus :

```
$ git checkout master
$ git checkout -b changements
$ echo -e "\nCeci n'est pas un pied de page" >> README.md
$ git add README.md
$ git commit -m "Ajout d'un pied de page rebelle"
```

Mais pendant que vous effectuez vos modifications, le dépôt officiel continue d'évoluer, et il est possible que des nouveautés arrivent sur la branche `master`. Les sorties des commandes `git fetch` et `git pull` vous en informeront.

Si c'est le cas, vous pourrez intégrer ces nouveautés à votre branche de développement avec les outils que nous avons vus précédemment. Un cycle complet de mise à jour de `master` et d'intégrations de ses nouveautés à votre branche pourrait ainsi ressembler à ceci :

```
$ git checkout master
$ git pull --ff-only
$ git checkout changements
$ git merge master
```

En cas de conflit de fusion, il vous appartiendra de vous adapter aux nouveautés de la branche `master` officielle qui sont incompatibles avec vos changements.

En règle général, plus une de vos branches a une durée de vie longue, plus il y a de chances que cela se produise. De plus, si vous utilisez `git rebase` plutôt que `git merge`, comme la politique de certains projets l'exige, il peut aussi arriver que vous rencontriez de conflits de fusion en cascade, où vous devrez faire face à des conflits non pas une seule fois, mais une fois par commit de votre branche !

Pour éviter cela et simplifier le passage en revue de vos changements par les mainteneurs, il est recommandé de privilégier des branches de durée de vie courte, aux objectifs précis et modestes, que l'on soumettra au dépôt officiel dès que possible. Nous allons maintenant étudier ce processus de soumission.

Mettre en ligne des changements sur son *fork*

Lorsque vous êtes satisfait de votre branche locale, vous pouvez la mettre en ligne sur votre *fork* avec la commande `git push`.

L'option `--set-upstream`, que nous avons évoquée précédemment, vous permettra de mémoriser l'association entre votre branche locale et la branche présente sur votre *fork*.

```
$ git push --set-upstream origin changements
```

Ainsi, vous pourrez par la suite vous contenter d'un simple `git push`.

Nous l'avons évoqué précédemment, dès lors que vous avez mis des changements en ligne sur un serveur, il faut être prudent avec les modifications d'historique et les `git push -f` associés pour ne pas « couper l'herbe sous le pied » aux gens qui travaillent par-dessus vos branches.

Mais l'un des avantages qu'il y a à utiliser un *fork*, c'est que ce dépôt n'appartient qu'à vous. Personne d'autre n'est donc censé baser son travail dessus, et c'est pourquoi la plupart des projets tolèrent l'utilisation libérale du *force-push* sur les *forks*, un confort très appréciable pour les amateurs d'édition d'historique.

Gardez cependant à l'esprit que cette règle de vie est à double tranchant. Si vous devez un jour baser votre travail sur le *fork* de quelqu'un d'autre, il vous appartiendra de vivre avec les suppressions de branches et autres modifications d'historique effectuées par cette personne sur son *fork*. Évitez donc si possible de baser du travail sur le *fork* d'autrui, en privilégiant plutôt le travail à partir du dépôt officiel du projet.

Soumission au dépôt officiel

Une fois que vous avez mis des changements en ligne sous la forme d'une branche sur votre *fork*, vous pouvez demander l'intégration de ces changements au dépôt officiel avec une fonctionnalité des forges git modernes, la *merge request*. Cette fonctionnalité est également appelée *pull request* par GitHub et BitBucket, mais comme nous utilisons GitLab dans ce TP, nous nous en tiendrons à sa terminologie afin d'éviter toute confusion.

Cette possibilité vous sera généralement directement offerte par l'affichage d'une URL dans la console lorsque vous poussez la branche sur votre *fork*, ainsi que par un bandeau dans l'interface web de votre *fork* sur GitLab pendant quelque temps après le *push*. Mais si vous avez manqué ces raccourcis, vous pouvez créer « manuellement » une *merge request* en allant dans l'interface web de votre *fork*, affichant la liste des branches, et cliquant sur le bouton « Merge request ».

Comme son nom l'indique, une *merge request* invite le · s mainteneur · s du dépôt officiel à intégrer des changements au projet en fusionnant les nouveautés de votre branche dans un historique officiel (généralement la branche `master`). Vous êtes invité à donner un titre court à votre soumission, ainsi qu'une description qui peut être plus étoffée et agrémentée de mise en page Markdown. Selon la forge que vous utilisez, d'autres fonctionnalités de gestion de projet peuvent vous être proposées lors de la création d'une *merge request*, nous vous invitons à consulter la documentation de votre forge pour en savoir plus.

Pour les mainteneurs, une *merge request* se présente sous la forme du titre et de la description que vous avez donnée, d'une visualisation des changements proposés, et d'une interface permettant de discuter desdits changements (pour proposer des modifications avant intégration par exemple).

Il est aussi possible pour le mainteneur de définir une batterie de tests automatisés que toute *merge request* doit passer avant d'être acceptée par le projet. On parle alors d'intégration continue.

Si des changements sont demandés par le mainteneur, vous pouvez ajouter ceux-ci à votre branche locale via de nouveaux commits et de les pousser sur votre *fork* avec un simple `git push`. La *merge request* associée sera alors mise à jour automatiquement.

À vous de jouer !

À ce stade, vous en savez assez pour commencer à vous amuser avec cette pauvre Déclaration des Droits de l'Homme.

Peut-être avez-vous toujours rêvé de paraphraser Olympe de Gouges en réécrivant entièrement le texte au féminin (« Déclaration des Droits de la Femme et de la Citoyenne ») ? Peut-être souhaitez-vous y ajouter l'article manquant qui résoudra enfin le débat séculaire entre pain au chocolat et chocolatine ? Ou bien peut-être est-il temps d'amender l'article 15 qui autorise chaque citoyen à demander des comptes aux agents publics ?

Modifiez le texte comme vous le souhaitez, en n'hésitant pas à créer des commits en cours de route pour vous permettre de revenir facilement en arrière. Puis, quand vous aurez fini, poussez votre branche finale sur votre *fork*, et soumettez une *merge request* au dépôt officiel pour terminer ce TP.

Conclusion

Dans ce dernier TP, nous avons vu comment utiliser git en réseau, avec une méthode de travail basée sur les *forks* et les *merge requests*.

Si ce n'est pas la seule manière d'utiliser git, cette méthode est une des plus populaires aujourd'hui, car elle sécurise le dépôt officiel, facilite les contributions extérieures, et évite certains problèmes liés à l'utilisation naïve d'un dépôt git

partagé (par exemple un *force-push* malencontreux sur une branche du dépôt principal qui sème le chaos dans le projet).

Nous terminerons cette formation par une exploration « à la carte » de sujets plus avancés, ou de tout point déjà abordé que vous souhaiteriez approfondir.

Exercices

1. Partez de votre clone du dépôt `ddhc`.
2. Vérifiez que les URLs de vos deux *remotes* sont correctes : `origin` doit pointer sur votre *fork*, `upstream` doit pointer sur le dépôt officiel, et les deux *remotes* doivent utiliser une connexion HTTPS.
3. Créez une nouvelle branche partant du `master`, et ajoutez-y un commit.
4. Poussez cette branche sur votre *fork*.
5. Ajoutez deux nouveaux commits à la branche, puis poussez-les sur votre *fork* aussi.
6. Mettez à jour votre dépôt vis-à-vis des éventuelles nouveautés du dépôt officiel du projet.
7. Détruisez la version locale de votre nouvelle branche avec `git branch -D`, puis régénérez-la à partir de la version sauvegardée sur votre dépôt.

Antisèche

Ce TP n'est qu'une mise en pratique des enseignements du TP4, et n'introduit aucune nouvelle commande git. Par conséquent, nous vous invitons à vous référer aux TPs précédents pour retrouver les commandes utilisées.

Informations complémentaires

Authentification SSH

Arrivé à la fin de ce TP, vous en avez probablement d'ors et déjà assez de taper votre mot de passe GitLab sans arrêt. Inutile, donc, de dire que vous ne vous imagineriez pas le faire tous les jours. Heureusement, des solutions existent.

Une méthode à la fois élégante et sécurisée pour vous authentifier sans mot de passe sur un dépôt git est d'utiliser le protocole à clé publique de SSH. L'idée est de générer une paire clé privée / clé publique comme vous le feriez pour vous authentifier sur un serveur Unix distant en SSH, puis de configurer la clé publique associée dans votre forge logicielle.

Une fois ceci fait, git se chargera ensuite d'envoyer à votre forge des commandes signées numériquement avec votre clé privée, et la forge pourra utiliser votre clé publique pour s'assurer que ces commandes viennent bien de vous et non d'un imposteur qui essaie de se faire passer pour vous.

Pour se passer totalement de mot de passe, votre clé privée doit être stockée de façon non chiffrée (pas de « passphrase »). Un tel stockage offre un niveau de sécurité acceptable lorsque vous respectez les prérequis suivants :

- Votre dossier personnel (« home ») est situé sur un stockage chiffré, et à l'abri du regard des autres utilisateurs si l'ordinateur est partagé.
- Vous n'exécutez que des logiciels de confiance, non susceptibles d'espionner votre clé privée et de la transmettre à des tiers malveillants.

Dans le cas contraire, une alternative intéressante peut être l'utilisation d'un « agent SSH », programme en tâche de fond qui mémorise le mot de passe de la clé SSH quelque temps après sa saisie dans le but de réduire la fréquence à laquelle on doit le saisir.

Une fois votre installation SSH locale configurée, il faut transmettre votre clé publique, qui vous identifie, à la forge logicielle que vous utilisez. La procédure de configuration exacte dépendra de la forge utilisée.

Après cela, vous pourrez effectuer vos clones ou configurer vos *remotes* avec les URL SSH à la place des URL HTTPS, et dire adieu à la saisie répétitive du mot de passe HTTPS !

Malheureusement, nous n'avons pas pu utiliser d'authentification SSH dans cette formation, car le pare-feu du campus de Gif-sur-Yvette bloque le trafic SSH sortant…

Petit tour d'horizon des forges logicielles

Dans ce TP, nous avons utilisé GitLab, mais il existe de nombreuses autres forges logicielles basées sur git.

Les trois moteurs de forge les plus utilisés à l'heure actuelle sont GitHub, GitLab et BitBucket. Les éditeurs de ces applications web proposent des instances hébergées sur leurs propres serveurs, selon un modèle commercial « freemium » (niveau de service gratuit limité, pour avoir plus il faut souscrire à un abonnement). Il est également possible d'en héberger une instance sur un serveur que l'on contrôle, par exemple lorsque le secret professionnel l'exige.

Dans ce cadre d'auto-hébergement, GitLab est le moteur le plus populaire dans le secteur académique. On peut mentionner, par exemple, des instances ayant des milliers d'utilisateurs au CERN, au centre de calcul de l'IN2P3 (CCIN2P3), ou au sein du réseau Mathrice (plate-forme PLM), ainsi que de nombreux déploiements plus modestes au sein de laboratoires individuels. La popularité de GitLab tient sans doute au fait que c'est l'éditeur de forge le plus ouvert à l'auto-hébergement.

Parmi les hébergeurs publics, GitHub domine. C'est donc souvent la meilleure solution lorsque l'on souhaite collaborer avec des développeurs hors de son cercle de sociabilité académique, ou lorsque l'on n'a pas accès à un hébergeur « en propre ». Notez toutefois que la légalité de l'utilisation de GitHub, hébergeur

américain, pour stocker des données de recherche européennes, est un sujet de polémique récurrent vis-à-vis duquel un consensus clair se fait toujours attendre.

Les moteurs de forge ci-dessus ont de nombreuses fonctionnalités en commun :

- Visualisation de l'historique et des différentes branches
- Parcours de l'arborescence des fichiers à la tête d'une branche donnée
- Visualisation de certains formats de fichier (ex : Markdown, code source)
- Possibilités limitées d'édition de fichiers en ligne
- Gestion de tickets (pour signaler des problèmes, recevoir des demandes…)
- Soumission et revue de changements (*pull requests*/*merge requests*)
- Documentation collaborative au format wiki
- Statistiques diverses et variées sur les contributions
- Outils de gestion de projet agile
- Systèmes de notification élaborées pour « suivre » l'évolution d'un projet

Cependant, chaque moteur a aussi ses spécificités :

- GitLab a toujours fourni en standard des fonctionnalités comme l'intégration continue ou l'hébergement d'images Docker, là où chez GitHub ce type de service a longtemps été délégué à des tierces parties comme Travis CI. Depuis fin 2019, il existe une solution GitHub « en propre », mais comme on pourrait s'y attendre elle est un peu moins mature que la concurrence au moment où sont écrites ces lignes (mi-2020).
- BitBucket propose une intégration poussée avec d'autres applications web du même éditeur (JIRA, Bamboo, Confluence, HipChat…)
- Le vocabulaire utilisé et le détail des statistiques proposées varie d'une forge à l'autre, et il n'est pas trivial de migrer l'ensemble d'un projet (y compris les tickets, merge requests…) d'une forge à une autre.